EXPERIMENT 3

D15A-57

Aim: Apply Decision Tree and Random Forest for classification tasks

Theory:

1. Dataset Source

Custom dataset : Link

2. Dataset Description

The dataset contains daily physical activity information collected from fitness tracking devices.

Features:

- Steps – Total number of steps taken per day
- Calories – Total calories burned per day
- Active Minutes – Sum of light, moderate, and very active minutes

Target Variable:

- Activity_Level (Low, Moderate, High)
    - Low: Steps < 5000
    - Moderate: 5000 ≤ Steps < 10000
    - High: Steps ≥ 10000

Dataset Characteristics:

- Type: Structured numerical dataset
- Size: Multiple daily records aggregated from CSV files
- Task Type: Multi-class classification

3. Mathematical Formulation of the Algorithm

A. Decision Tree

Decision Tree splits the dataset based on feature values to reduce impurity.

Gini Index Formula:

$Gini(D) = 1 - \Sigma (p_i)^2$

Where:

- pi = probability of class i
- c = number of classes

The best split is selected by minimizing Gini impurity.

The algorithm recursively partitions the data until:

- All samples belong to one class, or
- Maximum depth or stopping criteria is reached.

B. Random Forest

Random Forest is an ensemble method combining multiple Decision Trees.

It works using:

1. Bootstrap Sampling
   Random subsets of training data are created.

2. Random Feature Selection
   At each split, a random subset of features is selected.

3. Majority Voting (Classification)
   Final prediction is determined by majority vote of all trees.

4. Algorithm Limitations

Decision Tree Limitations:

- Prone to overfitting
- Sensitive to small changes in data
- May create biased trees for imbalanced datasets

Random Forest Limitations:

- Less interpretable than a single Decision Tree
- Higher computational cost
- Requires more memory

5. Methodology / Workflow

The experiment followed the below workflow:

1. Data Collection – Load multiple CSV files

2. Data Preprocessing

    - Convert timestamps
    - Aggregate daily values

3. Feature Engineering

    - Calculate total active minutes

4. Target Variable Creation

    - Classify activity level based on step count

5. Train-Test Split

6. Model Training

    - Decision Tree
    - Random Forest

7. Model Evaluation

    - Accuracy
    - Precision
    - Confusion Matrix
    - Feature Importance

8. Visualization

    - Decision Tree structure
    - Confusion matrix graph

6. Performance Analysis

The model performance was evaluated using:

- Accuracy

- Precision (Macro Average)
- Confusion Matrix
- Classification Report
- Feature Importance (Gini Importance)

Decision Tree provided clear interpretability but showed moderate variance.
Random Forest achieved higher accuracy and better generalization due to ensemble learning.

Feature importance analysis showed that “Steps” contributed the most towards predicting activity level.

7. Hyperparameter Tuning

Hyperparameter tuning was performed to improve model performance

Grid Search / Manual tuning was used to test multiple combinations of parameters.
Tuning improved accuracy and reduced overfitting compared to default settings.

Decision Tree
Code:

```python
import matplotlib
matplotlib.use("Agg")

import pandas as pd
import glob
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, classification_report, confusion_matrix

steps_df = pd.concat([pd.read_csv(f) for f in glob.glob("steps*.csv")], ignore_index=True)
calories_df = pd.concat([pd.read_csv(f) for f in glob.glob("calories*.csv")], ignore_index=True)
active_df = pd.concat([pd.read_csv(f) for f in glob.glob("active_minutes*.csv")], ignore_index=True)

for df in [steps_df, calories_df, active_df]:
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['date'] = df['timestamp'].dt.date

steps_daily = steps_df.groupby('date')['steps'].sum().reset_index()
calories_daily = calories_df.groupby('date')['calories'].sum().reset_index()

active_df['active_minutes'] = (
    active_df['light'] +
    active_df['moderate'] +
    active_df['very']
)
active_daily = active_df.groupby('date')['active_minutes'].sum().reset_index()

df = steps_daily.merge(calories_daily, on='date')
df = df.merge(active_daily, on='date')

def classify_activity(steps):
    if steps < 5000:
        return "Low"
```

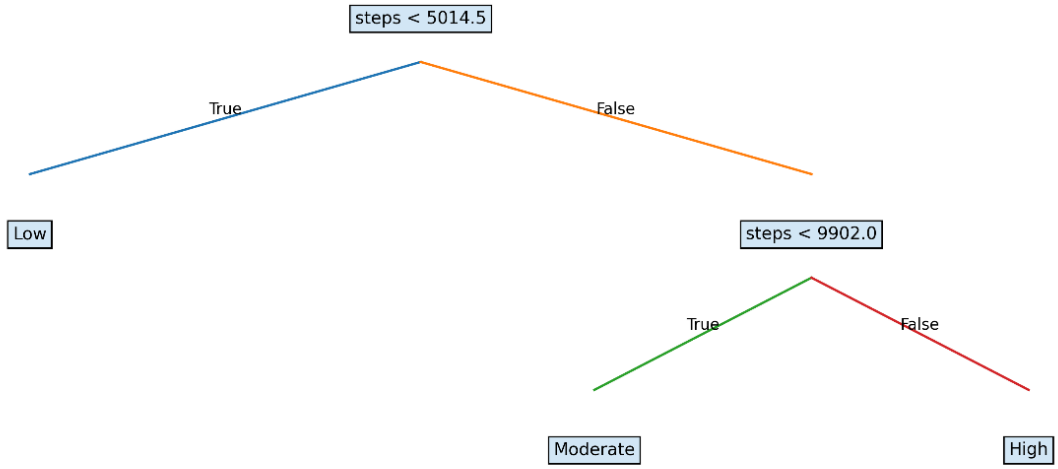
```python
    elif steps < 10000:
        return "Moderate"
    else:
        return "High"

df['Activity_Level'] = df['steps'].apply(classify_activity)

X = df[['steps', 'calories', 'active_minutes']]
y = df['Activity_Level']

le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)


y_pred = model.predict(X_test)

print("\n===== MODEL PERFORMANCE =====")

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')

print(f"\nAccuracy: {accuracy:.4f}")
print(f"Precision (Macro): {precision:.4f}")

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

print("\nClassification Report:")
```

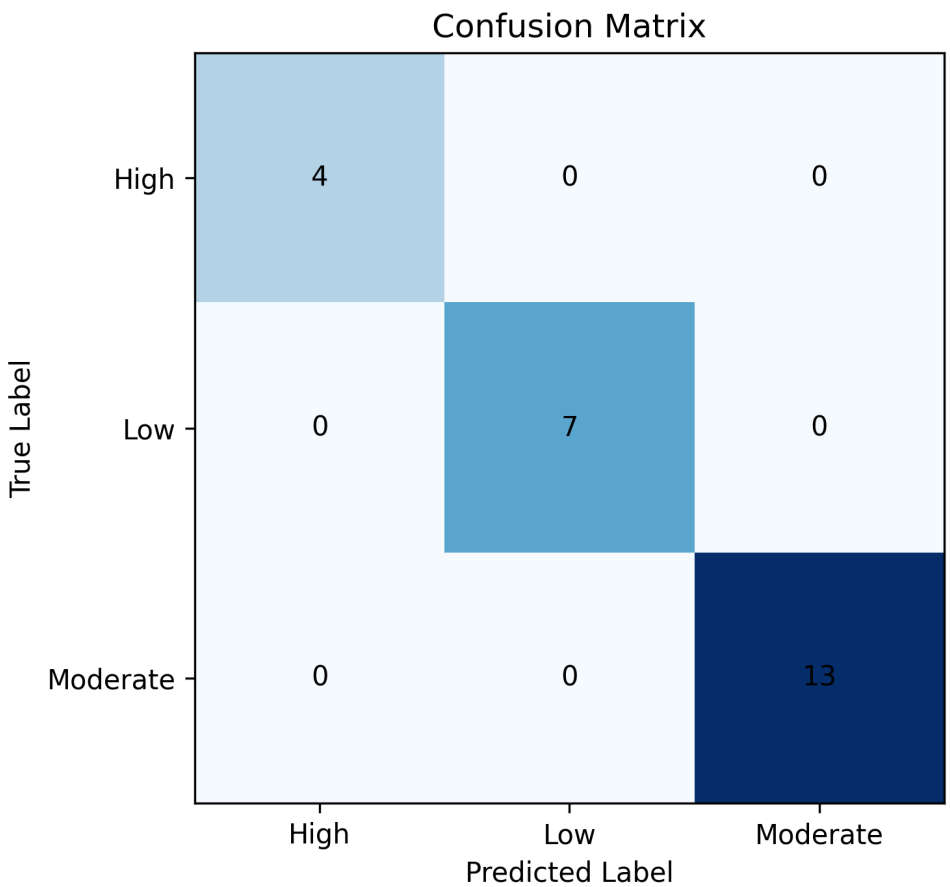
```python
print(classification_report(y_test, y_pred, target_names=le.classes_))
print("\n===== FEATURE IMPORTANCE (Gini Importance) =====")
for name, importance in zip(X.columns, model.feature_importances_):
    print(f"{name} : {importance:.4f}")


print("\n===== GINI VALUES PER NODE =====")
tree = model.tree_
for i in range(tree.node_count):
    print(f"Node {i} Gini: {tree.impurity[i]:.4f}")

plt.figure(figsize=(18, 10))
ax = plt.gca()
ax.set_axis_off()

tree = model.tree_
feature_names = X.columns
class_names = le.classes_

def draw_node(node, x, y, dx):


    if tree.children_left[node] == -1:
        class_id = np.argmax(tree.value[node])
        text = class_names[class_id]
    else:
        feature = feature_names[tree.feature[node]]
        threshold = round(tree.threshold[node], 2)
        text = f"{feature} < {threshold}"

    ax.text(
        x, y, text,
        ha='center',
        va='center',
        bbox=dict(
            boxstyle="square",
            facecolor="#D6EAF8",
```

```python
            edgecolor="black"
        ),
        fontsize=11
    )

    if tree.children_left[node] != -1:
        left = tree.children_left[node]
        ax.plot([x, x - dx], [y - 0.05, y - 0.18])
        ax.text(x - dx / 2, y - 0.11, "True", ha='center')
        draw_node(left, x - dx, y - 0.25, dx / 1.8)


    if tree.children_right[node] != -1:
        right = tree.children_right[node]
        ax.plot([x, x + dx], [y - 0.05, y - 0.18])
        ax.text(x + dx / 2, y - 0.11, "False", ha='center')
        draw_node(right, x + dx, y - 0.25, dx / 1.8)

draw_node(0, 0.5, 0.9, 0.25)

plt.xlim(0, 1)
plt.ylim(0, 1)

plt.savefig("decision_tree_clean.png", dpi=300)
print("\nDecision tree saved as decision_tree_clean.png")

plt.figure()

plt.imshow(cm, cmap="Blues")

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.xticks(np.arange(len(le.classes_)), le.classes_)
plt.yticks(np.arange(len(le.classes_)), le.classes_)
```

```python

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=300)

print("Confusion matrix graph saved as confusion_matrix.png")
```

Output:


steps < 5014.5
True
False
Low
steps < 9902.0
True
False
Moderate
High


Confusion Matrix
High
Low
Moderate
True Label
4
0
0
0
7
0
0
0
13
High
Low
Moderate
Predicted Label

Random Forest:

Code:

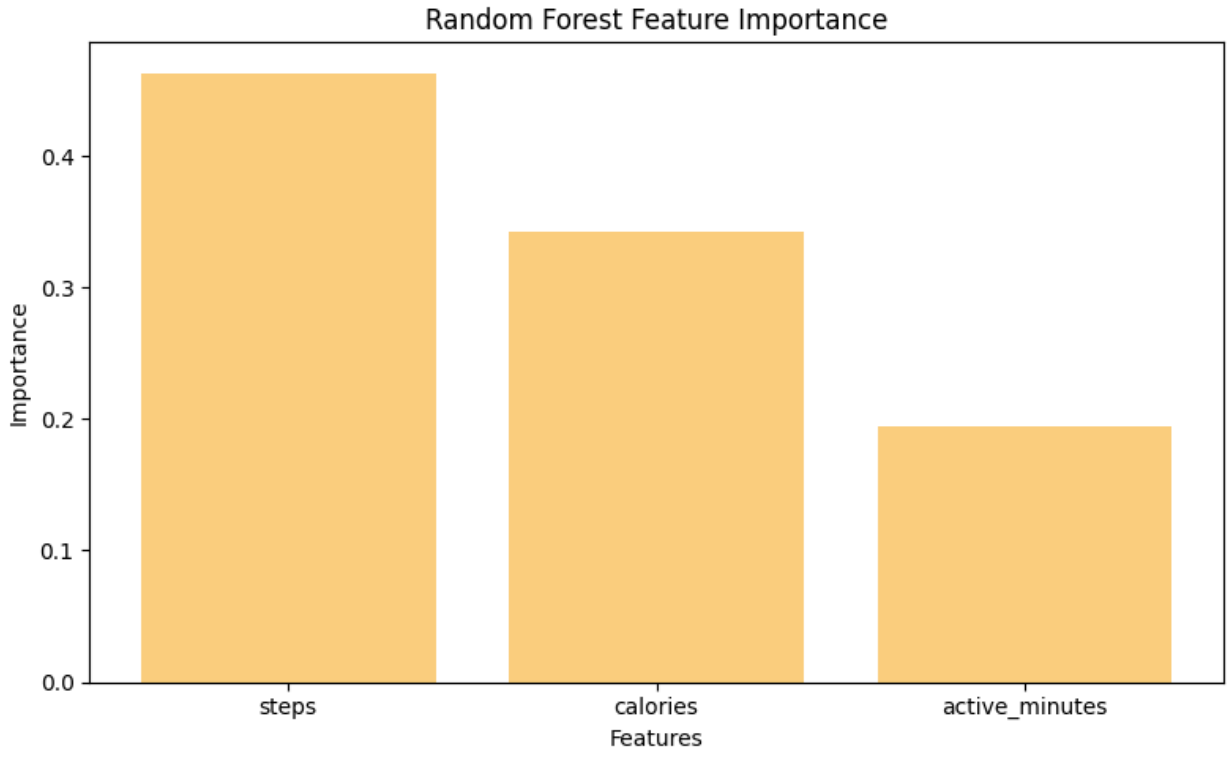
```python
import pandas as pd
import glob
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder


steps_df = pd.concat([pd.read_csv(f) for f in glob.glob("steps*.csv")], ignore_index=True)
calories_df = pd.concat([pd.read_csv(f) for f in glob.glob("calories*.csv")], ignore_index=True)
active_df = pd.concat([pd.read_csv(f) for f in glob.glob("active_minutes*.csv")], ignore_index=True)

for df in [steps_df, calories_df, active_df]:
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['date'] = df['timestamp'].dt.date


steps_daily = steps_df.groupby('date')['steps'].sum().reset_index()
calories_daily = calories_df.groupby('date')['calories'].sum().reset_index()

active_df['active_minutes'] = (
    active_df['light'] +
    active_df['moderate'] +
    active_df['very']
)

active_daily = active_df.groupby('date')['active_minutes'].sum().reset_index()

df = steps_daily.merge(calories_daily, on='date')
```

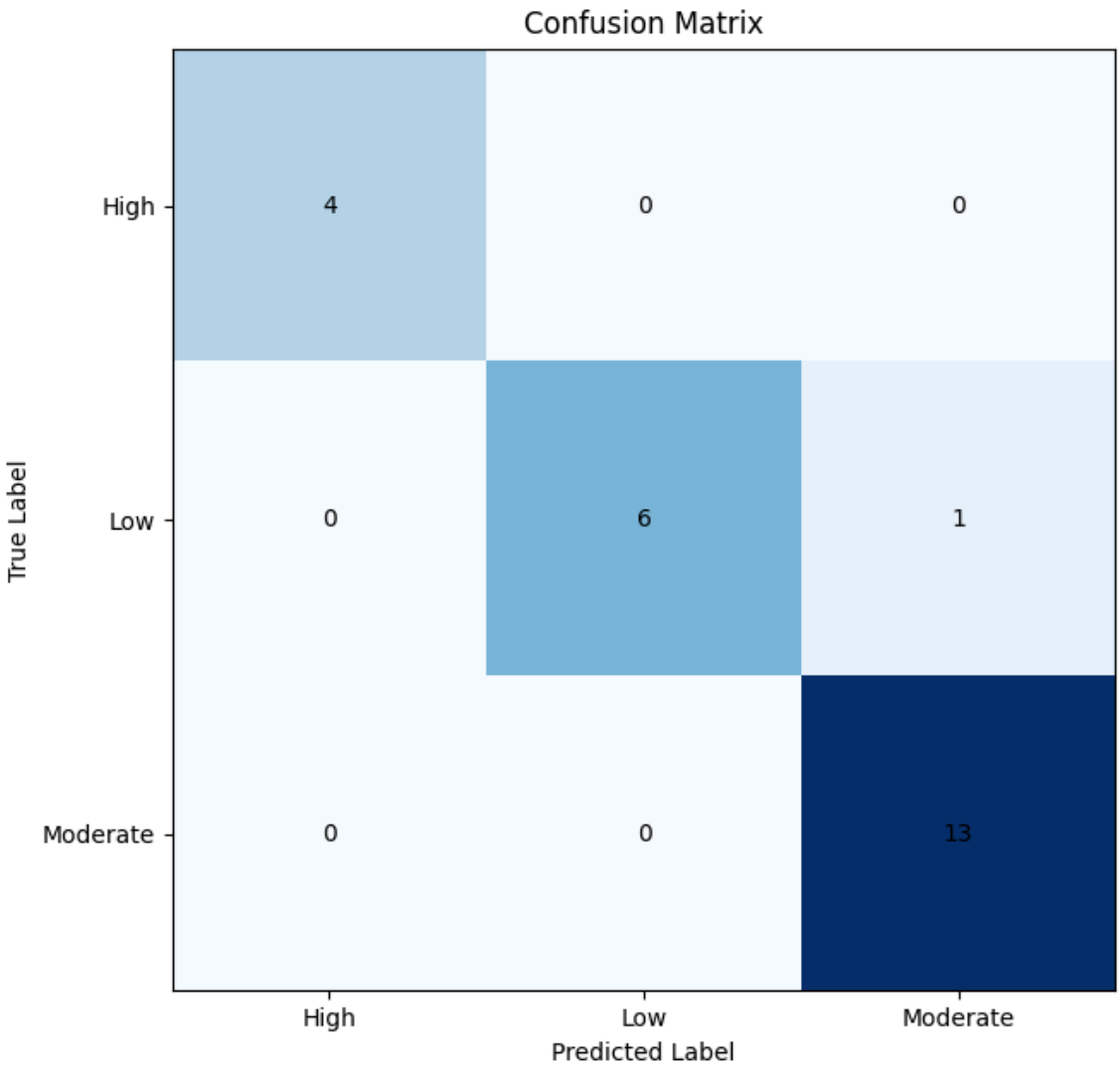
```python
df = df.merge(active_daily, on='date')

def classify_activity(steps):
    if steps < 5000:
        return "Low"
    elif steps < 10000:
        return "Moderate"
    else:
        return "High"

df['Activity_Level'] = df['steps'].apply(classify_activity)

X = df[['steps', 'calories', 'active_minutes']]
y = df['Activity_Level']

le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42
)

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

```

```python
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=le.classes_))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)


plt.figure(figsize=(6,6))
plt.imshow(cm, cmap="BLUES")
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

classes = le.classes_
plt.xticks(np.arange(len(classes)), classes)
plt.yticks(np.arange(len(classes)), classes)

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j],
                 ha="center",
                 va="center",
                 color="black")

plt.tight_layout()
```

```python
plt.show()

importances = rf_model.feature_importances_
features = X.columns

plt.figure(figsize=(8,5))
plt.bar(features, importances, color="#fad07d")
plt.title("Random Forest Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.tight_layout()
plt.show()
```

Output:


Random Forest Feature Importance
Importance
0.0
0.1
0.2
0.3
0.4
steps
calories
active_minutes
Features



Conclusion:

Decision Tree and Random Forest are powerful supervised learning algorithms widely used for classification and regression tasks.

A Decision Tree is useful because it is simple, easy to interpret, and visually understandable. It clearly shows how decisions are made through feature-based conditions. This makes it highly suitable for problems where model transparency and explainability are important, such as healthcare analysis, business decision-making, and academic projects. However, a single Decision Tree may overfit the data if not properly controlled.

Random Forest improves upon Decision Trees by combining multiple trees to produce more accurate and stable predictions. By using bootstrap sampling and random feature selection, it reduces overfitting and increases generalization performance. It is especially useful for complex datasets where higher accuracy and robustness are required.